



Pontifícia Universidade Católica de Minas Gerais

Instituto de Ciências Exatas e Informática

AED III - Trabalho Prático

Grupo 5

Componentes do Grupo: Karen Cristina Satiro da Costa, Marina Veríssimo Almeida Campidelli, Paula Maria Alves de Oliveira.

1. Descrição do Problema

O projeto aborda a gestão de dados nutricionais e culinários, focando na organização de alimentos e na elaboração de receitas personalizadas. O problema central é a necessidade de um sistema que armazene de forma persistente e eficiente as informações de usuários (clientes), ingredientes (alimentos) e o modo de preparo (receitas). A solução utiliza arquivos binários para garantir que os dados sejam preservados e recuperados rapidamente, permitindo que cada cliente gerencie sua própria conta e suas criações culinárias.

2. Objetivo do Trabalho

Desenvolver um sistema de persistência em memória secundária utilizando arquivos binários.

Implementar um CRUD completo com controle de cabeçalho (ID) e exclusão lógica (lápide) para as entidades Cliente, Alimento e Receita.

Estabelecer a base para relacionamentos 1:N (um cliente para várias receitas) e N:N (receitas e alimentos).

Garantir que tipos de dados variados, como strings multivaloradas (categorias) e datas (criação de conta), sejam manipulados corretamente via `RandomAccessFile`.

3. Requisitos Funcionais

RF01: Cadastrar, consultar, atualizar e excluir Clientes, incluindo a data de criação da conta.

RF02: Cadastrar, consultar, atualizar e excluir Alimentos com suporte a múltiplas categorias.

RF03: Cadastrar, consultar, atualizar e excluir Receitas (Título, Informações, Tempo e Porção).

RF04: Permitir que um Cliente gerencie múltiplas receitas próprias (Relacionamento 1:N).

RF05: Possibilitar a associação de diversos alimentos a uma única receita (Relacionamento N:N).

RF06: Realizar buscas por ID ou por campos textuais (como Nome do Alimento ou Título da Receita).

4. Requisitos Não Funcionais

RNF01: A persistência deve ser feita estritamente em arquivos binários (sem uso de SGBDs).

RNF02: O sistema deve reaproveitar ou identificar espaços de registros excluídos através do byte de "lápide".

RNF03: Uso obrigatório da linguagem Java e manipulação de bytes via `DataOutputStream` /

DataStream.

RNF04: O acesso aos dados deve ser otimizado através de um cabeçalho que armazena o último ID válido.

5. Atores

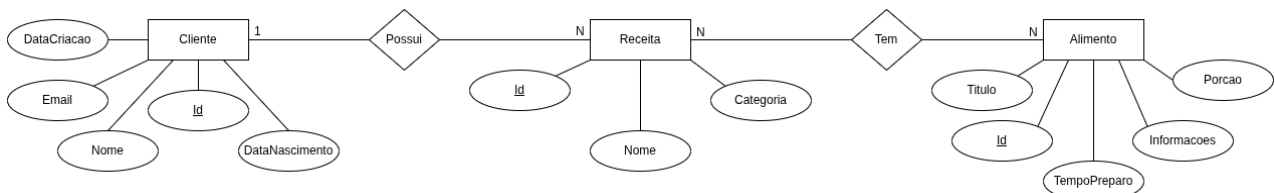
Cliente: O usuário principal que realiza o próprio cadastro, gerencia seus dados (incluindo data de criação), alimentos e cria suas receitas.

Administrador: Responsável pela manutenção.

6. Diagrama de Caso de Uso



7. Diagrama Entidade-Relacionamento



8. Arquitetura Proposta (Representação gráfica obrigatória.) O sistema seguirá o padrão MVC +

DAO, onde:

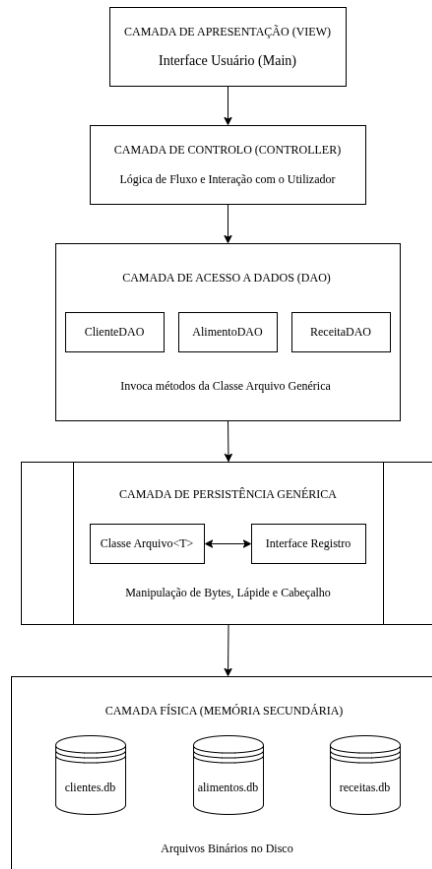
Registro (Interface): Interface que obriga os modelos a terem os métodos `toByteArray` e `fromByteArray`.

Arquivo<T> (Classe Genérica): Responsável por abrir o `RandomAccessFile`, ler/escrever o cabeçalho de IDs e realizar o posicionamento de ponteiros para as operações de CRUD.

DAOs Específicos (Ex: ReceitaDAO): Classes que utilizam o `Arquivo` genérico e implementam buscas específicas através de iteração no arquivo binário.

Armazenamento: Arquivos físicos .db que armazenam os registros em sequência de bytes.

Diagrama de Arquitetura em Camadas



9. Formulário

a) Os registros foram armazenados em arquivos binários utilizando *RandomAccessFile*. Cada registro possui lápide, tamanho e dados. A serialização é feita através do *toByteArray()* e *fromByteArray()*, e implementados pelas entidades usando *DataOutputStream* e *DataInputStream*.

b) Os atributos multivalorados foram implementados como arrays de `String(String[])`. Primeiro é gravado o tamanho o tamanho do array, com cada string sendo gravado usando *writeUTF()*. Depois lê-se a quantidade, cria um array e preenche com *readUTF()*.

c) A exclusão lógica foi implementada utilizando uma lápide no início de cada registro. A lápide é marcada como “*” e o espaço é adicionado a uma lista encadeada de espaços livres.

d) Foram utilizadas chaves de busca não indexadas como: nome do alimento, categoria, título da receita, tempo de preparo, nome.

e) Foi utilizada uma hash extensível para a indexação da chave primária(ID). A estrutura é composta por um diretório. A função hash define o bucket de destino, e em caso de overflow ocorre divisão (split), com possível expansão do diretório. Esse mecanismo proporciona buscas eficientes, sendo utilizado apenas para a chave primária, enquanto as demais consultas são feitas de forma sequencial.

f) O relacionamento 1:N foi implementado através do uso de IDs, de forma lógica. Busca-se o registro principal via ID(hash), depois para acessar os relacionados percorre-se os registros e filtrar por chave estrangeira.

g) Os índices hash são persistidos em arquivos binários separados em: arquivo de diretório e arquivo de buckets. A cada inserção/remoção o bucket é atualizado diretamente no arquivo. Os dados e índices são mantidos consistentes: sempre que um registro é criado, o índice é atualizado; sempre que excluído, o índice remove a chave.

h) O projeto está organizado em camadas:

Model: classes de domínio e serialização.

DAO: escapsula operações CRUD.

Util: estrutura de arquivos e índices.

View: interação com o usuário

10. Repositório GitHub

<https://github.com/PKaaa/TP---AEDs-III>

11. Link Vídeo

<https://youtu.be/rhRQ3qb2DfE>